Kiến trúc đa nền tảng Web và Desktop (Tauri) và cập nhật tự động — Thiết kế chi tiết

StructPaper v2. Ngày lập: 08/11/2026. Phạm vi: (A) cách tổ chức mã để cùng một giao diện chạy trên web và trên ứng dụng desktop Tauri với các tính năng khác nhau theo nền tảng; (B) cơ chế thông báo và cập nhật tự động cho bản desktop. Liên quan: tài liệu Đề xuất kiến trúc & design pattern, tài liệu Biên dịch LaTeX trong trình duyệt. Những chi tiết dựa vào hành vi của Tauri v2 hay công cụ bên thứ ba cần được xác nhận lại với phiên bản đang dùng khi triển khai. Phần thiết kế trải nghiệm (UI/UX) của từng khác biệt giữa web và desktop không nằm trong tài liệu này.

0. Tóm tắt điều hành

Hiện trạng. Ứng dụng đã có bản desktop Tauri; quy trình phát hành chạy trên GitHub Actions (đẩy tag thì tạo release). Web và desktop dùng chung mã giao diện.

Vấn đề. (1) Có tính năng chỉ có ở một nền tảng hoặc được thay bằng tính năng khác (ví dụ web có nút tải bản máy tính, desktop thay bằng kiểm tra cập nhật), và sẽ còn nhiều trường hợp tương tự. Nếu rải điều kiện kiểu `if (isTauri)` khắp nơi thì mã sẽ rối và khó kiểm thử. (2) Người dùng desktop không tự nhận bản mới như web, cần cơ chế thông báo và cập nhật. (3) Web, desktop và backend có nhịp phát hành khác nhau nên dễ lệch phiên bản.

Tám quyết định chính:

1. **Mô tả theo ý định, không theo công nghệ.** Giao diện chỉ nói *lưu tài liệu*, *kiểm tra phiên bản*, *biên dịch*; mỗi nền tảng cung cấp cách thực hiện qua cổng `PlatformServices`. Mã giao diện không biết Tauri.
2. **Một nơi duy nhất nhận diện nền tảng** (composition root). Hai adapter `platform-web` và `platform-desktop`; chỉ gói desktop được import `@tauri-apps/*`, ép bằng công cụ kiểm ranh giới.
3. **Khác biệt giao diện đi qua "khe" (slot)** có kiểu bắt buộc khai báo cho mọi nền tảng; một tính năng được bật khi **cả ba** điều kiện đúng: nền tảng hỗ trợ, cờ từ xa cho phép, người dùng có quyền.
4. **Bản desktop là giao diện tƒ©nh**: không có route của Next.js; mọi lời gọi API đi qua `HttpClient` có `baseUrl`.

5. **Lệch phiên bản** giữa web, desktop và backend được quản bằng API có phiên bản, `minClientVersion` và `schemaVersion` của dữ liệu.
6. **Cập nhật bằng plugin updater của Tauri**, chữ ký bắt buộc. Giai đoạn đầu dùng tệp `latest.json` trên GitHub Releases; chuyển sang endpoint động ở Spring Boot khi cần kênh beta, phát hành dần, ép cập nhật hay thu hồi một bản lỗi.
7. **Phát hành bằng GitHub Actions**: tag → build theo ma trận → release **nháp** → kiểm tra → publish. Kênh stable và beta tách theo dạng tag.
8. **Trải nghiệm cập nhật không chặn người dùng và bảo vệ dữ liệu** (autosave trước khi cài, sao lưu trước migration, không hỗ trợ hạ cấp), và dùng chung giao diện `UpdateService` cho web (tải lại trang) và desktop (cài bản mới).

---

1. Bối cảnh và mục tiêu

1.1 Hiện trạng

- Bản web chạy trên trình duyệt, cập nhật bằng cách triển khai.
- Bản desktop dùng Tauri, đóng gói cùng giao diện; Tectonic được đóng kèm để biên dịch cục bộ.
- Phát hành: GitHub Actions, đẩy tag thì tạo release trên GitHub.
- Backend: Spring Boot + MySQL (đang hoặc sẽ có), dùng chung cho cả hai nền tảng.

1.2 Vấn đề cần giải quyết

#	Vấn đề	Hệ quả nếu để nguyên
P1	Tính năng khác nhau theo nền tảng, và sẽ thêm dần	Điều kiện nền tảng rải rác, khó đọc, khó test; dễ quên một nền tảng khi thêm tính năng; mã web vô tình kéo theo thư viện Tauri
P2	Bản desktop là giao diện đã đóng gói	Một số thứ của Next.js (route, SSR) không tồn tại, gây lỗi chỉ xuất hiện ở desktop
P3	Người dùng desktop không tự có bản mới	Nhiều người chạy bản cũ, gây tốn hỗ trợ và lỗi tương thích với backend

#	Vấn đề	Hệ quả nếu để nguyên
P4	Lệch phiên bản web, desktop, backend	Dữ liệu lưu bằng bản mới mở không được bằng bản cũ, hoặc bản cũ gọi API đã đổi
P5	Phát hành thủ công dễ sai	Phiên bản không khớp giữa tag, cấu hình và bộ cài; khóa ký làm mất hoặc lộ

1.3 Mục tiêu

- **G1.** Thêm một tính năng chỉ có ở một nền tảng, hoặc có hai cách thực hiện khác nhau, mà không phải sửa mã giao diện chung.
- **G2.** Compiler sẽ báo lỗi khi thiếu bản triển khai cho một nền tảng nào đó.
- **G3.** Bản web không chứa mã của Tauri; bản desktop không phụ thuộc route của Next.js.
- **G4.** Người dùng desktop được thông báo khi có bản mới và cập nhật được trong vài thao tác, không mất dữ liệu.
- **G5.** Bản cập nhật được xác minh bằng chữ ký; khóa ký được bảo vệ và có kế hoạch dự phòng.
- **G6.** Backend có thể ép bản quá cũ phải cập nhật, và có thể chặn một bản lỗi mà không cần phát hành lại ứng dụng.
- **G7.** Quy trình phát hành lặp lại được, có kiểm tra phiên bản tự động và bước duyệt trước khi người dùng nhận bản mới.

1.4 Phi mục tiêu

- Không thiết kế UI/UX chi tiết cho từng khác biệt web và desktop; việc đó do nhóm giao diện quyết định.
- Không làm bản di động (Tauri mobile) trong giai đoạn này, nhưng kiến trúc không cản trở việc thêm sau.
- Không xây hệ thống phân phối riêng ngoài GitHub và Spring Boot ở giai đoạn đầu.
- Không hạ cấp tự động về phiên bản cũ khi có lỗi; xử lý bằng phát hành bản sửa hoặc chỉ định phiên bản từ server động (mục 3.4).

1.5 Thuật ngữ

Thuật ngữ	Ý nghĩa
Capability	Năng lực mà một nền tảng có (ví dụ có updater, có hệ tệp)
Adapter	Bản triển khai một cổng cho một nền tảng cụ thể
Slot (khe)	Vị trí trong giao diện mà nội dung do nền tảng quyết định
Kênh (channel)	Luồng phát hành: `stable` (người dùng thường) và `beta` (nhóm thử nghiệm)
Bản cập nhật	Gói mà updater tải về và cài, kèm chữ ký

---

2. Kiến trúc đa nền tảng

2.1 Nguyên tắc

1. **Ý định ở trên, công nghệ ở dưới.** Giao diện gọi `files.save(doc)`, không gọi `tauri.fs` hay `<a download>`.
2. **Một nơi nhận diện nền tảng.** Chỉ composition root biết chạy trên web hay desktop; toàn bộ phần còn lại nhận `PlatformServices` từ context.
3. **Thêm nền tảng hoặc tính năng khác biệt không sửa mã giao diện dùng chung** (Open/Closed). Thiếu bản triển khai cho một nền tảng là lỗi biên dịch.
4. **Ranh giới phụ thuộc rõ ràng.** Mã web không import Tauri; mã desktop không phụ thuộc Next.js server.
5. **Thử được mà không cần Tauri.** Mọi thành phần giao diện chạy được với một nền tảng giả trong test.
6. **Quyền tối thiểu.** Bản desktop chỉ được cấp những quyền Tauri thật sự cần.

2.2 Cổng nền tảng (`packages/platform`)

```
type PlatformKind = 'web' | 'desktop';

interface Capabilities {
  kind: PlatformKind;
```

```
  hasAppUpdater: boolean;       // desktop: true
  hasNativeCompiler: boolean;   // desktop: true (Tectonic đóng kèm)
  hasFileSystem: boolean;       // desktop: true
  canInstallApp: boolean;       // web: true (gợi ý tải bản máy tính);
desktop: false
}

interface RuntimeConfig {
  apiBaseUrl: string;           // web: cùng origin hoặc cấu hình; desktop:
URL backend
  appVersion: string;           // bản đang chạy
  buildId: string;
  channel: 'stable' | 'beta';
}

interface PlatformServices {
  capabilities: Capabilities;
  config: RuntimeConfig;
  files: FileService;
  updates: UpdateService;
  shell: ShellService;
  http: HttpClient;
  auth: AuthService;
  compilers: CompilerPort[];    // adapter biên dịch khả dụng trên nền tảng
này
}
```

Dịch vụ	Việc	Adapter web	Adapter desktop
`FileService`	Lưu, mở, chọn tệp; trả về `FileHandle` trừu tượng (không lộ `File` hay đường dẫn)	Tải xuống, chọn tệp bằng `input`	Hộp thoại hệ thống và đọc ghi tệp trong phạm vi được cấp quyền
`UpdateService`	Kiểm tra, cài, khởi động lại (mục 3.6)	Phát hiện bản web mới, gợi ý tải lại	Plugin updater của Tauri
`CompilerPort`	Biện dịch LaTeX (xem tài liệu biên dịch)	`WasmCompiler`, `RemoteCompiler`	`NativeCompiler` (Tectonic đóng kèm), `RemoteCompiler`

Dịch vụ	Việc	Adapter web	Adapter desktop
`ShellService`	Mở liên kết ngoài, thao tác cửa sổ, tiêu đề	`window.open`	Plugin shell/opener, API cửa sổ
`HttpClient`	Gọi backend, gắn token, thử lại	`fetch` tới `apiBaseUrl`	`fetch` hoặc plugin HTTP tới `apiBaseUrl`
`AuthService`	Đăng nhập, làm mới token, đăng xuất	Chuyển hướng trang	Mở trình duyệt hệ thống, nhận kết quả qua deep link hoặc cổng loopback; lưu token an toàn

Mỗi dịch vụ là một interface nhỏ với một mục đích rõ, không gộp thành một “PlatformUtils” lớn.

2.3 Cấu trúc gói và ranh giới

```
packages/
  platform/            # interface, Capabilities, FeatureId, slot registry
(không phụ thuộc nền tảng)
  platform-web/        # adapter web
  platform-desktop/    # adapter desktop; dùng @tauri-apps/*
apps/
  web/                 # Next.js (có route, SSR nếu dùng)
  desktop/             # vỏ Tauri (src-tauri) và cấu hình đóng gói giao diện
tƒ©nh
```

Quy tắc phụ thuộc (kiểm bằng `dependency-cruiser`, vi phạm thì fail CI):

Quy tắc	Nội dung
R1	Chỉ `platform-desktop` và `apps/desktop` được import `@tauri-apps/*`
R2	`platform` không import `platform-web` hay `platform-desktop`
R3	Gói giao diện (`feature-*`, `ui-kit`, `editor-core`) chỉ phụ thuộc `platform`, không phụ thuộc adapter

Quy tắc	Nội dung
R4	`platform-desktop` không import mã chỉ chạy phía server của Next.js
R5	Chiều ngược lại: `apps/web` không import `platform-desktop` trừ qua nhánh nạp động có điều kiện

Adapter desktop được nạp bằng `import()` động, nhờ đó bản web không đóng gói mã Tauri.

2.4 Nhận diện nền tảng và composition root

```
// apps/*/platform-bootstrap.ts — nơi duy nhất biết nền tảng
export async function createPlatform(): Promise<PlatformServices> {
  if (await isRunningInTauri()) {
    const { createDesktopPlatform } = await import('@structpaper/platform-
desktop');
    return createDesktopPlatform();
  }
  const { createWebPlatform } = await import('@structpaper/platform-web');
  return createWebPlatform();
}
```

- Dùng hàm kiểm tra do thư viện Tauri cung cấp (nếu có trong phiên bản đang dùng) thay vì tự đoán bằng tên biến toàn cục, vì tên đó có thể đổi giữa các phiên bản.
- Việc nhận diện chỉ chạy ở phía client. Tránh nhận diện trong lúc render phía server để không gây lệch hydration; hiển thị trạng thái chờ cho tới khi nền tảng sẵn sàng.
- Có thêm cờ lúc build (`APP_TARGET=web` hoặc `desktop`) để tree-shake và để kiểm tra: bản dựng sai nền tảng phải báo lỗi sớm.
- Trong test, `createFakePlatform({ kind, overrides })` thay cho hàm trên.

```
const platform = await createPlatform();
root.render(
  <PlatformProvider value={platform}>
    <App />
  </PlatformProvider>
);
```

2.5 Khe giao diện, năng lực và cờ từ xa

Khe (slot). Một vị trí giao diện có nội dung khác nhau theo nền tảng được đăng ký một lần:

```
// package: platform
const slots = defineSlots({
  'app-distribution': { web: DownloadDesktopButton, desktop:
CheckForUpdateButton },
  'file-actions':     { web: WebFileActions,        desktop:
DesktopFileActions },
  'compile-engine':   { web: null,                  desktop:
NativeEngineBadge }, // null = ẩn
});

<Slot id='app-distribution' />   // tự chọn theo platform.capabilities.kind
```

Kiểu của mỗi khe là `Record<PlatformKind, Component | null>`: thêm nền tảng mới vào `PlatformKind` hoặc thêm khe mà thiếu một bản sẽ **lỗi biên dịch**. `null` là quyết định rõ "ẩn trên nền tảng này", không phải quên.

Ba thứ, ba vai trò khác nhau. Dễ nhầm lẫn nên cần quy ước:

Công cụ	Dùng khi	Ví dụ
Adapter (`PlatformServices`)	Cùng một ý định, cách thực hiện khác nhau, giao diện dùng chung	Lưu tài liệu, biên dịch
Slot	Nội dung giao diện khác nhau tại cùng một vị trí	Nút tải bản máy tính và nút kiểm tra cập nhật
Cờ năng lực (`isEnabled(featureId)`)	Một tính năng có thể bật hay tắt độc lập với giao diện	Chức năng thử nghiệm, tính năng theo vai trò

Điều kiện bật một tính năng là phép AND của ba nguồn:

```
function isEnabled(feature: FeatureId, ctx: FeatureContext): boolean {
  return ctx.platform.supports(feature)      // nền tảng có năng lực
      && ctx.remoteFlags.allows(feature)     // cờ từ xa từ backend cho phép
      && ctx.user.canUse(feature);           // vai trò và quyền của người
dùng
}
```

Cờ từ xa cho phép tắt một tính năng lỗi trên một nền tảng mà không phát hành lại ứng dụng (đặc biệt quan trọng với bản desktop vì người dùng chậm cập nhật).

2.6 Ràng buộc kỹ thuật của bản desktop

Bản desktop đóng gói giao diện đã dựng sẵn, nên có những người chơi "quen dùng trên web" sẽ không còn:

1. **Giao diện tƒ©nh, không có server Next.js.** Bản desktop dùng xuất tƒ©nh (`output: 'export'` hoặc tương đương). Các route `app/api/...`, middleware, SSR không tồn tại; một số tính năng như tối ưu ảnh mặc định cần cấu hình khác (cần kiểm lại theo phiên bản Next đang dùng). Mọi thứ từng chạy trong route (ví dụ proxy AI, compile giai đoạn đầu) phải đi qua backend thật.
2. **Gọi API qua `HttpClient` có `baseUrl`.** Không dùng đường dẫn tương đối kiểu `/api/...` trong mã giao diện chung. `baseUrl` lấy từ `RuntimeConfig`, khác nhau cho từng môi trường.
3. **CORS và CSP.** Origin của WebView desktop khác origin web và khác nhau giữa hệ điều hành; backend phải liệt kê các origin này trong cấu hình CORS, và cấu hình CSP của Tauri phải cho phép `connect-src` tới backend.
4. **Xác thực.** Cookie của bên thứ ba có thể bị chặn trong WebView; ưu tiên token (OAuth với PKCE) và lưu token bằng kho khóa an toàn của hệ điều hành qua plugin phù hợp, không lưu trong `localStorage`.
5. **Quyền của Tauri v2.** Mỗi lệnh và plugin mà giao diện được phép gọi phải được khai báo trong cấu hình quyền của ứng dụng. Nguyên tắc quyền tối thiểu: chỉ bật updater, khởi động lại, hộp thoại tệp, hệ tệp **trong phạm vi thư mục cụ thể**, và chạy chương trình ngoài **chỉ cho `tectonic` với đối số được giới hạn** (sidecar).
6. **Lưu trữ.** IndexedDB chạy trong WebView và nằm trong thư mục dữ liệu của ứng dụng; vị trí và hành vi khác nhau giữa hệ điều hành. Không dựa vào `localStorage` cho dữ liệu quan trọng; có sao lưu ra tệp trước khi migration (mục 3.8).
7. **Liên kết, cửa sổ, kéo thả.** `window.open` và `target=_blank` phải đi qua `ShellService`; kéo thả tệp trả về `FileHandle` trừu tượng chứ không lộ đường dẫn hay đối tượng `File` ra giao diện.
8. **Engine hiển thị khác nhau.** Tauri dùng WebView của hệ điều hành (Chromium-dựa trên Windows, WebKit trên macOS và Linux), nên CSS, IME tiếng Việt và một số API có thể khác; phải kiểm thử trên từng hệ điều hành được hỗ trợ (mục 2.9).
9. **Service Worker.** Bản desktop không đăng ký Service Worker; việc làm nóng cache và cập nhật do `UpdateService` và bộ cài đảm nhiệm.

2.7 Biên dịch trên desktop: `NativeCompiler`

`NativeCompiler` là một adapter của `CompilerPort` (cùng cổng với `WasmCompiler` và `RemoteCompiler` ở tài liệu biên dịch), chạy Tectonic được đóng kèm ứng dụng (sidecar). Giao diện và Editor không đổi; chỉ `PlatformServices.compilers` của desktop có thêm một phần tử.

Khía cạnh	Thiết kế
Cách gọi	Plugin shell của Tauri chạy sidecar `tectonic` với **đối số cố định** (danh sách cho phép), không ghép chuỗi lệnh. Không bật shell-escape
Thư mục làm việc	Mỗi lần biên dịch một thư mục tạm riêng trong thư mục cache của ứng dụng; ghi `main.tex`, ảnh, `.cls`, font vào đó; đọc PDF và log ra; dọn sau khi xong
Hủy và giới hạn	Hủy bằng cách dừng tiến trình; có đồng hồ canh thời gian tối đa; mỗi lúc chỉ một tiến trình biên dịch cho mỗi bản thảo
Ngoại tuyến	Tectonic mặc định tải tệp hỗ trợ theo yêu cầu từ một bundle trên mạng. Muốn chạy ngoại tuyến thật phải đóng sẵn hoặc làm nóng cache bundle, và **ghim đúng phiên bản bundle**
Tên tệp sidecar	Tauri yêu cầu tệp thực thi đi kèm đặt tên theo hệ nền tảng (hậu tố target triple); mỗi hệ điều hành và kiến trúc một tệp. Xác nhận quy ước theo phiên bản Tauri đang dùng
Báo cáo	Log đi qua `log-parser` dùng chung để cho ra `CompileReport` giống các adapter khác
Môi trường	Ghi nhận là `TexEnvironment` loại `desktop-native` (thêm vào danh sách loại ở tài liệu biên dịch): phiên bản Tectonic, bundle, bộ font, lớp `.cls` đều có hash
Bộ định tuyến	Có `hasNativeCompiler` và môi trường sẵn sàng thì bản nháp ưu tiên `NativeCompiler`; bản chính thức (PDF/X, prepress) vẫn ở server như trước

Khía cạnh	Thiết kế
Đồng bộ với server	Phiên bản Tectonic trong bộ cài và trên server phải cùng bản và cùng bundle; mỗi lần nâng Tectonic phải nâng cả hai trong cùng chu kỳ phát hành và chạy bộ so sánh độ lệch (mục 10 tài liệu biên dịch) cho môi trường `desktop-native`

Phiên bản Tectonic gắn với phiên bản ứng dụng: người dùng chưa cập nhật app thì vẫn dùng Tectonic cũ. Vì vậy bản thảo ghim môi trường (`ProfilePin.environmentIds`) và cơ chế cập nhật (mục 3) phối hợp với nhau; nếu bản thảo ghim một môi trường mà app hiện tại không có, hệ thống đề nghị cập nhật hoặc dùng server.

2.8 Lệch phiên bản giữa web, desktop và backend

Web cập nhật ngay khi triển khai, desktop cập nhật theo nhịp phát hành và theo việc người dùng có chịu cập nhật hay không, nên luôn tồn tại nhiều phiên bản desktop cùng lúc. Có bốn trục cần quản:

Trục	Cơ chế	Chi tiết
API	Phiên bản trong đường dẫn (`/api/v1`) và quy tắc *mở rộng trước, thu hẹp sau*	Thêm trường không phá vỡ; thay đổi phá vỡ thì chạy `/v2` song song trong một khoảng thời gian cam kết (ví dụ phiên bản desktop trong 90 ngày gần nhất hoặc N bản gần nhất). Chỉ gỡ bản cũ sau khi số liệu cho thấy gần như không còn ai dùng
Phiên bản tối thiểu	`GET /api/v1/config/client` trả `minClientVersion`, `recommendedVersion`, cờ tính năng, chế độ bảo trì; backend trả HTTP 426 (cần nâng cấp) kèm tiêu đề `X-Min-Client-Version` khi bản quá cũ	Client kiểm tra lúc khởi động và khi nhận 426; bản dưới mức tối thiểu hiện màn hình bắt buộc cập nhật (mục 3.6)

Trục	Cơ chế	Chi tiết
Dữ liệu	`schemaVersion` của bản thảo, và `minWriterVersion` ghi cùng bản thảo	Client **từ chối mở** bản thảo có `schemaVersion` cao hơn mức nó hiểu. Server **từ chối ghi** khi phiên bản client thấp hơn `minWriterVersion` của bản thảo, tránh bản cũ ghi đè và làm mất trường mới
Tính năng	Thương lượng năng lực: `config/client` trả danh sách `features` mà server hỗ trợ	Client ẩn tính năng mà server chưa hỗ trợ và ngược lại; không giả định mọi phiên bản biết mọi thứ

Thay đổi phá vỡ phải làm hai bước:

1. Triển khai backend hỗ trợ **cả cũ lẫn mới**; kiểm tra các phiên bản client được hỗ trợ.
2. Phát hành client mới; chờ tỷ lệ cập nhật đủ cao; nâng `minClientVersion`; gỡ đường cũ ở backend.

Bảng tương thích tối thiểu cần giữ (và kiểm thử, mục 2.9):

Client	Backend	Kỳ vọng
Web mới nhất	Backend mới nhất	Đầy đủ
Desktop trong khoảng được hỗ trợ	Backend mới nhất	Dùng được; tính năng mới có thể ẩn
Desktop dưới `minClientVersion`	Backend mới nhất	Chặn bằng màn hình bắt buộc cập nhật
Bất kỳ client	Bản thảo có `schemaVersion` cao hơn	Từ chối mở, đề nghị cập nhật

2.9 Kiểm thử đa nền tảng

Loại	Nội dung
Đơn vị	Dùng `FakePlatform`; mỗi khe và mỗi tính năng có test cho cả hai `kind`

Loại	Nội dung
Hợp đồng cổng	Một bộ test dùng chung cho mỗi cổng (ví dụ `fileServiceContract`) chạy với adapter web (DOM giả) và adapter desktop (lệnh Tauri giả). Hai adapter phải cùng đạt
Kiểu biên dịch	`Record<PlatformKind, ...>` ở khe và dịch vụ; `tsc` trong CI nên thiếu bản triển khai là lỗi
Ranh giới	`dependency-cruiser` (mục 2.3) và một bước phân tích gói web để **khẳng định bản web không chứa `@tauri-apps`**
E2E web	Playwright
E2E desktop	Kiểm thử khói trên vỏ Tauri thật bằng WebDriver của Tauri: mở app, mở và lưu tệp, biên dịch bằng sidecar, kiểm tra cập nhật với server giả
Ma trận hệ điều hành	Windows (các bản được hỗ trợ), macOS (Intel và Apple Silicon), Linux nếu hỗ trợ. Gõ tiếng Việt (Telex, VNI) kiểm thủ công trên từng nền tảng vì engine hiển thị khác nhau
Tương thích phiên bản	Chạy bộ kiểm thử hợp đồng API của client được hỗ trợ cũ nhất với backend mới nhất
Cập nhật	Xem mục 3.12

2.10 Quy trình thêm một tính năng chỉ có ở một nền tảng

1. Đặt tên `FeatureId` và viết ý định bằng một câu (không nhắc công nghệ).
2. Chọn công cụ theo bảng ở mục 2.5: adapter, slot hay cờ năng lực.
3. Khai báo trong `packages/platform` (kiểu, khe).
4. Viết bản triển khai cho **cả hai** `kind`, hoặc ghi `null` có chủ đích.
5. Nếu dùng thêm plugin hoặc lệnh Tauri: thêm đúng quyền tối thiểu vào cấu hình quyền, có người xem xét về bảo mật.
6. Viết test hợp đồng hoặc test khe; chạy trên ma trận.
7. CI **sinh tự động** bảng *tính năng × nền tảng* từ registry (đưa vào tài liệu của nhóm giao diện); đổi bảng này phải đi cùng thay đổi mã.

8. Mẫu pull request có hộp kiểm: đã xử lý web, đã xử lý desktop, đã cập nhật bảng quyền, đã cập nhật ghi chú phát hành nếu người dùng thấy sự khác biệt.

---

3. Cập nhật tự động

3.1 Phạm vi

- **Bản desktop:** kiểm tra, thông báo, tải, cài và khởi động lại bằng plugin updater của Tauri.
- **Bản web:** không có bước cài; cập nhật là triển khai. Vẫn có thông báo *tải lại để dùng bản mới* qua cùng giao diện `UpdateService` (mục 3.7) để trải nghiệm thống nhất.
- Ngoài phạm vi: cập nhật tự động của Tectonic riêng lẻ (đi cùng bản cập nhật ứng dụng, mục 2.7), cập nhật di động.

3.2 Cơ chế updater của Tauri v2

Thành phần	Vai trò
Plugin updater (phần Rust và phần JavaScript)	Gọi `check()` để hỏi nguồn cập nhật; nếu có bản mới thì trả thông tin gồm phiên bản, ngày và ghi chú; `downloadAndInstall` tải và cài, có hàm báo tiến độ
Plugin process	Khởi động lại ứng dụng sau khi cài
Cặp khóa ký của updater	Khóa riêng ký bản cập nhật lúc build; khóa công khai nằm trong cấu hình ứng dụng và dùng để xác minh trước khi cài. **Chữ ký là bắt buộc**, không tắt được
Cấu hình `plugins.updater`	`pubkey` (nội dung khóa công khai, không phải đường dẫn tệp), `endpoints` (một hay nhiều URL), tùy chọn riêng cho Windows như `installMode`
Cấu hình `bundle.createUpdaterArtifacts`	Bật thì lúc build sinh ra bản cập nhật kèm chữ ký

Thành phần	Vai trò
Biến trong URL	`{{target}}`, `{{arch}}`, `{{current_version}}` để server biết ứng dụng nào đang hỏi

Trạng thái các tệp do build sinh ra (tên và định dạng cụ thể phụ thuộc phiên bản Tauri):

Hệ điều hành	Bản cập nhật	Chữ ký
Windows	Bộ cài NSIS hoặc MSI (hoặc bản nén của chúng ở các phiên bản Tauri cũ hơn)	Tệp `.sig` đi kèm
macOS	`.app.tar.gz`	Tệp `.sig` đi kèm
Linux	AppImage	Tệp `.sig` đi kèm

Tùy chọn `installMode` trên Windows quyết định cách cài: có thể là kiểu thụ động (cửa sổ nhỏ có thanh tiến độ, không cần tương tác), kiểu giao diện cơ bản hoặc kiểu im lặng. Xác nhận các giá trị hợp lệ theo phiên bản đang dùng.

3.3 Khóa ký và chữ ký của hệ điều hành

Hai thứ này **khác nhau** và đều cần:

	Khóa ký updater	Chữ ký phần mềm của hệ điều hành
Mục đích	Ứng dụng đã cài xác minh bản cập nhật là của bạn	Hệ điều hành tin ứng dụng khi cài và chạy
Tạo ở đâu	Lệnh `tauri signer generate` của Tauri CLI	Windows: chứng chỉ ký mã hoặc dịch vụ ký đám mây. macOS: chứng chỉ Developer ID và **notarize** với Apple
Lưu ở đâu	Secrets của GitHub Actions (biến môi trường lúc build; tệp `.env` không dùng được)	Secrets của GitHub Actions
Nếu thiếu	Không phát hành được bản cập nhật hợp lệ	Windows hiện cảnh báo SmartScreen; macOS có thể chặn không cho mở

Quản lý khóa updater:

- Khóa riêng chỉ năm ở secrets của môi trường phát hành, tách khỏi secrets thường; cấu hình **môi trường GitHub có người duyệt bắt buộc** cho job dùng khóa.
- **Sao lưu ở hai nơi an toàn, ngoại tuyến** (trình quản lý bí mật của tổ chức và một bản in hoặc ổ khóa). **Mất khóa riêng nghĩa là người dùng đã cài app không nhận được cập nhật nữa** vì họ chỉ tin khóa công khai cũ; chỉ còn cách yêu cầu họ cài lại bằng tay.
- Ghi rõ ai có quyền truy cập và quy trình khi nhân sự thay đổi.
- **Xoay khóa** (khi nghi ngờ lộ): cần một bản phát hành chuyển tiếp mang khóa công khai mới và được ký bằng khóa cũ; sau khi phần lớn đã cập nhật mới chuyển sang ký bằng khóa mới. Quy trình này phải được thử trước trên môi trường thử, không đợi đến lúc cần.

> Cần xác định: nhóm đang dùng những hệ điều hành nào cho người dùng (Windows, macOS, Linux) và đã có chứng chỉ ký mã chưa. Mục 3.13 liệt kê việc cần chốt.

3.4 Nguồn cập nhật

Lựa chọn	Mô tả	Khi nào dùng
A. Tệp tĩnh `latest.json` trên GitHub Releases	Endpoint trỏ tới `latest.json` của release mới nhất. Công cụ `tauri-action` có thể sinh và đính kèm tệp này khi build	**Bắt đầu**: đơn giản, khớp quy trình tag → release
B. Server động (endpoint trong Spring Boot)	Server trả 204 khi không có bản mới, 200 kèm JSON khi có; có thể chọn phiên bản theo kênh, theo tỷ lệ phát hành dần, theo danh sách chặn	Khi cần kênh beta, phát hành dần, chặn một bản lỗi, ép nâng cấp
C. Kho đối tượng hoặc CDN + tệp tĩnh	Giống A nhưng không phụ thuộc GitHub	Khi repo phát hành là riêng tư hoặc cần băng thông riêng

Lưu ý quan trọng về repo riêng tư. Ứng dụng của người dùng không tải được tệp từ release của repo riêng tư nếu không có xác thực, và không nên nhúng token vào ứng dụng. Dùng một repo công khai chỉ chứa bản phát hành, hoặc đặt bản cập nhật ở nơi công khai rồi dùng lựa chọn B hay C.

Mẫu `latest.json` (lựa chọn A):

```
{
  'version': '1.4.0',
  'notes': 'Nội dung thay đổi bằng tiếng Việt',
  'pub_date': '2026-11-08T10:00:00Z',
  'platforms': {
    'windows-x86_64':  { 'signature': '...', 'url':
'https://.../StructPaper_1.4.0_x64-setup.exe' },
    'darwin-aarch64':  { 'signature': '...', 'url':
'https://.../StructPaper_aarch64.app.tar.gz' },
    'darwin-x86_64':   { 'signature': '...', 'url':
'https://.../StructPaper_x64.app.tar.gz' },
    'linux-x86_64':    { 'signature': '...', 'url':
'https://.../StructPaper_1.4.0_amd64.AppImage' }
  }
}
```

(Mẫu dùng dấu nháy đơn để hiển thị; tệp thật là JSON hợp lệ với nháy kép. Khóa nền tảng có dạng *hệ điều hành-kiến trúc*.)

Hợp đồng endpoint động (lựa chọn B):

Mục	Nội dung
Yêu cầu	`GET /api/v1/desktop/updates/{channel}/{target}/{arch}/{currentVersion}` kèm tiêu đề `X-Install-Id`
Không có bản mới	HTTP 204, không có thân
Có bản mới	HTTP 200 với JSON gồm `version`, `notes`, `pub_date`, `url`, `signature` (đúng định dạng mà updater yêu cầu)
Chọn phiên bản	Theo kênh; theo `rolloutPercent` (băm `X-Install-Id` ra một thùng cố định); bỏ qua các phiên bản trong danh sách chặn; có thể trả bản cũ hơn để quay lui nếu client được cấu hình chấp nhận (mục 3.11)
Logic ép nâng cấp	**Không** nhét vào JSON của updater. Dùng `GET /api/v1/config/client` (mục 2.8)

Mục	Nội dung
Lưu trữ	Metadata phát hành trong MySQL (`desktop_releases`: version, channel, state, rollout_percent, notes, created_at); tệp thật ở GitHub Releases hoặc kho đối tượng
`X-Install-Id`	UUID ngẫu nhiên sinh một lần khi cài, lưu cục bộ; **không** gắn với danh tính người dùng; chỉ dùng để chia thùng phát hành dần và thống kê

Kênh phát hành: updater cho phép đặt URL cập nhật lúc chạy, nên chọn kênh (`stable` hay `beta`) được đưa vào URL hoặc tiêu đề; kênh lưu trong cài đặt người dùng và `RuntimeConfig`. Việc đổi endpoint theo kênh lúc chạy làm ở phía Rust (bộ dựng updater) hoặc để server quyết định theo tham số; xác nhận cách làm với phiên bản đang dùng.

3.5 Quy trình phát hành CI/CD

Quy ước phiên bản và tag

- Phiên bản theo SemVer: `vMAJOR.MINOR.PATCH`; bản thử nghiệm `vMAJOR.MINOR.PATCH-beta.N`.
- **Một nguồn sự thật cho phiên bản.** Một script đồng bộ `package.json`, `tauri.conf.json` và `Cargo.toml`; bước kiểm tra trong CI so sánh với tag và **fail nếu lệch**.

Các giai đoạn của workflow

#	Giai đoạn	Việc	Kết quả
1	Kiểm tra	Định dạng tag đúng; phiên bản các tệp khớp tag; có ghi chú phát hành; lint và test chạy xanh	Chặn sớm các sai sót thông thường
2	Build theo ma trận	Windows, macOS Apple Silicon, macOS Intel, Linux; mỗi dòng build giao diện tĩnh rồi đóng gói Tauri kèm Tectonic đúng nền tảng	Bộ cài và bản cập nhật cho từng nền tảng

#	Giai đoạn	Việc	Kết quả
3	Ký	Ký bản cập nhật bằng khóa updater; ký mã theo hệ điều hành (và notarize ở macOS)	Tệp `.sig` và gói đã ký
4	Tạo release nháp	`tauri-action` tạo **release nháp**, tải các tệp lên và sinh `latest.json`	Release chưa hiện với người dùng
5	Kiểm tra sau build	Mỗi bộ cài có `.sig`; `latest.json` đủ các nền tảng; kích thước và băm hợp lý; chữ ký xác minh được bằng khóa công khai	Tránh phát hành bản thiếu
6	Duyệt	Người phụ trách thử bản nháp trên máy thật (đặc biệt cập nhật từ bản trước lên bản này), rồi bấm duyệt môi trường phát hành	Chốt chặn của con người
7	Publish	Chuyển release nháp thành chính thức (bản beta đánh dấu pre-release)	Từ lúc này người dùng mới nhận được

Giai đoạn 4 và 7 tách rời là chốt an toàn: bản nháp chưa được endpoint `latest` trỏ tới, nên có thể kiểm thử thoải mái trước khi công bố.

Mẫu workflow (khung tham khảo)

```
name: release
on:
  push:
    tags: ['v*']

permissions:
  contents: read

jobs:
  verify:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@<commit-sha>
      - run: node scripts/check-version.mjs ${{ github.ref_name }}   # tag
phải khớp phiên bản
```

```
      - run: pnpm install --frozen-lockfile && pnpm lint && pnpm test

  build:
    needs: verify
    environment: release              # môi trường có người duyệt bắt buộc,
chứa secrets ký
    permissions:
      contents: write                 # chỉ job này được tạo release
    strategy:
      fail-fast: false
      matrix:
        include:
          - { os: windows-latest, args: '' }
          - { os: macos-latest,   args: '--target aarch64-apple-darwin' }
          - { os: macos-latest,   args: '--target x86_64-apple-darwin' }
          - { os: ubuntu-22.04,   args: '' }
    runs-on: ${{ matrix.os }}
    steps:
      - uses: actions/checkout@<commit-sha>
      - uses: pnpm/action-setup@<commit-sha>
      - uses: dtolnay/rust-toolchain@<commit-sha>
      - run: pnpm install --frozen-lockfile
      - uses: tauri-apps/tauri-action@<commit-sha>
        env:
          GITHUB_TOKEN: ${{ secrets.GITHUB_TOKEN }}
          TAURI_SIGNING_PRIVATE_KEY: ${{ secrets.TAURI_SIGNING_PRIVATE_KEY }}
          TAURI_SIGNING_PRIVATE_KEY_PASSWORD: ${{
secrets.TAURI_SIGNING_PRIVATE_KEY_PASSWORD }}
          # thêm secrets ký mã của Windows và macOS (chứng chỉ, notarize)
        with:
          tagName: ${{ github.ref_name }}
          releaseName: 'StructPaper ${{ github.ref_name }}'
          releaseBody: 'Xem ghi chú phát hành'
          releaseDraft: true
          prerelease: ${{ contains(github.ref_name, '-beta') }}
          args: ${{ matrix.args }}
```

Đây là khung để định hướng. Tên tham số của `tauri-action`, cách ký trên từng hệ điều hành và phiên bản các action phải được kiểm tra lại theo phiên bản đang dùng. **Ghim mọi action theo mã commit** (thay `<commit-sha>`) thay vì tên nhánh hay thẻ, để tránh bị thay đổi ngầm.

Ghi chú phát hành

- Viết bằng tiếng Việt, ngắn, theo mẫu cố định: *Tính năng mới*, *Cải thiện*, *Sửa lỗi*, *Lưu ý khi cập nhật*.
- Có thể soạn tự động từ nhãn của pull request (công cụ như release-drafter), rồi người phụ trách chỉnh lại trước khi publish.
- Nội dung này chính là `notes` mà ứng dụng hiển thị khi báo cập nhật (mục 3.6); giới hạn độ dài và chỉ dùng tập con Markdown đơn giản để hiển thị an toàn.

Kênh stable và beta

Trên GitHub, liên kết `latest` chỉ trỏ tới release chính thức mới nhất, **không** gồm bản pre-release. Vì vậy kênh beta không thể dùng `latest.json` của GitHub theo cùng cách. Hai hướng:

- Giữ một release pre-release cố định (ví dụ tên `beta-channel`) và cập nhật tệp `latest.json` của nó mỗi lần phát hành beta.
- Hoặc dùng endpoint động (lựa chọn B) để chọn theo kênh. Đây là lý do cần B khi muốn có beta thật sự.

3.6 Trải nghiệm trong ứng dụng và `UpdateService`

Giao diện dịch vụ

```
interface UpdateInfo {
  version: string;
  notes?: string;                  // lấy từ ghi chú phát hành
  publishedAt?: string;
  mandatory: boolean;              // do minClientVersion quyết định (mục
2.8), không do updater
  kind: 'app-binary' | 'web-reload';
}

type UpdateState =
  | { s: 'idle' }
  | { s: 'checking' }
  | { s: 'available'; info: UpdateInfo }
  | { s: 'downloading'; info: UpdateInfo; received: number; total?: number }
  | { s: 'ready'; info: UpdateInfo }           // đã tải và xác minh, chờ cài
hoặc khởi động lại
  | { s: 'installing'; info: UpdateInfo }
  | { s: 'error'; code: UpdateErrorCode; retryAt?: number };
```

```
interface UpdateService {
  readonly channel: 'stable' | 'beta';
  state$: Observable<UpdateState>;
  check(opts?: { manual?: boolean }): Promise<UpdateInfo | null>;   // null =
đang là bản mới nhất
  install(info: UpdateInfo): Promise<void>;                        // chạy các
móc trước khi cài rồi cài
  restart(): Promise<void>;
  skipVersion(version: string): void;
  remindLater(hours: number): void;
}
```

Giao diện thông báo (`UpdateBanner` , hộp thoại ghi chú, màn hình bắt buộc cập nhật) chỉ đọc `state$` và gọi các phương thức trên. Hình thức hiển thị do nhóm UI/UX quyết định; tài liệu này chỉ quy định hành vi và các ràng buộc.

Máy trạng thái

```
 idle ──check()──▶ checking ──không có bản mới──▶ idle
                       │
                       ├──có bản mới──▶ available ──install()──▶ downloading
──▶ ready ──restart()──▶ (khởi động lại)
                       │                  | skipVersion / remindLater
|
                       |                  ▼
▼
                       |                idle
error ──retry──▶ checking
                       └──lỗi mạng──▶ idle (im lặng) hoặc error (khi người
dùng bấm kiểm tra)
```

Chính sách kiểm tra

Tình huống	Hành vi
Khởi động ứng dụng	Kiểm tra sau một khoảng trễ ngắn (khoảng 10–30 giây), không chặn việc mở app
Định kỳ	Mỗi vài giờ kèm nhiễu ngẫu nhiên (jitter) để không dồn yêu cầu về server cùng lúc

Tình huống	Hành vi
Khi có mạng trở lại	Kiểm tra nếu lần cuối đã quá một khoảng nhất định
Người dùng bấm *Kiểm tra cập nhật*	Kiểm tra ngay và luôn trả kết quả rõ ràng, kể cả *đang là bản mới nhất* hoặc *không kết nối được*
Kiểm tra tự động thất bại	Im lặng, thử lại với thời gian giãn cách tăng dần
Server trả `minClientVersion` cao hơn bản hiện tại hoặc HTTP 426	Chuyển sang trạng thái bắt buộc ngay

Quy tắc hành vi (ràng buộc cho giao diện)

1. **Không chăn đường.** Thông báo bản thường không được che tài liệu đang soạn và không giành quyền nhập liệu. Không hiện hộp thoại khi người dùng đang gõ hoặc đang biên dịch.
2. **Đủ lựa chọn:** xem nội dung thay đổi, cập nhật ngay, nhắc sau (ghi thời điểm nhắc), bỏ qua bản này (ghi phiên bản đã bỏ qua; vẫn thông báo bản mới hơn).
3. **Bản bắt buộc** (do `minClientVersion`): chỉ khi bản hiện tại không còn được backend hỗ trợ. Màn hình chặn nhưng vẫn cho **xuất bản sao lưu** dữ liệu cục bộ trước khi cập nhật.
4. **Hiển thị ghi chú phát hành** an toàn: chỉ dùng tập con Markdown, không chạy HTML thô.
5. **Hiển thị tiến độ** khi tải; cho hủy được trước khi cài.
6. **Không tự khởi động lại bất ngờ.** Cài xong thì báo và đề nghị khởi động lại, hoặc khởi động lại khi người dùng đồng ý.
7. **Sau khi cập nhật:** ở lần mở đầu tiên của bản mới, hiện một lần thông tin *đã cập nhật lên X* cùng ghi chú.

Các móc trước khi cài (`beforeInstall`)

Chạy tuần tự, bất kỳ móc nào thất bại thì **không cài** và báo người dùng:

Thứ tự	Móc	Việc
1	`flushAutosave`	Ghi mọi thay đổi chưa lưu xuống IndexedDB, chờ xác nhận ghi xong
2	`settleCompile`	Chờ biên dịch đang chạy kết thúc, hoặc hủy nếu người dùng đồng ý
3	`flushOutbox`	Cố gắng đẩy hàng chờ đồng bộ lên server; không thành công thì vẫn cài vì dữ liệu nằm an toàn cục bộ
4	`createBackup`	Tạo bản sao lưu trước khi cài (mục 3.8)
5	`closeDialogs`	Đóng hộp thoại, giải phóng khóa tệp

Trên Windows, trình cài đặt sẽ đóng ứng dụng khi cài; các móc phải xong **trước** khi gọi cài.

Xử lý lỗi

Mã	Tình huống	Hành vi
`U1_OFFLINE`	Không có mạng	Kiểm tra tự động: im lặng. Người dùng bấm kiểm tra: báo không kết nối được
`U2_SIGNATURE`	Chữ ký không hợp lệ	**Không cài**, báo lỗi bảo mật, ghi nhận để điều tra; đây có thể là dấu hiệu tệp bị thay đổi
`U3_DOWNLOAD`	Tải gián đoạn hoặc lỗi	Cho thử lại; không cài gói chưa tải đủ
`U4_DISK`	Hết dung lượng	Báo rõ dung lượng cần thiết
`U5_PERMISSION`	Không đủ quyền ghi vào thư mục cài (cài theo máy trên Windows)	Hướng dẫn tải bộ cài mới; cân nhắc bộ cài theo người dùng hiện tại (mục 3.9)
`U6_ENDPOINT`	Nguồn trả dữ liệu không hợp lệ	Ghi log, thử lại sau, không báo lỗi cho người dùng nếu kiểm tra tự động

Mã	Tình huống	Hành vi
`U7_INSTALL`	Cài thất bại giữa chừng	Giữ bản cũ nếu được, báo và hướng dẫn cài tay; gửi mã lỗi (không kèm nội dung bài)

3.7 Phía web: cùng giao diện, adapter khác

Bản web không cài gì, nhưng người dùng đang mở một tab lâu có thể đang chạy mã cũ khi bản mới đã được triển khai. Adapter web của `UpdateService` có `kind: 'web-reload'`:

Cách phát hiện	Mô tả
Tệp phiên bản	Máy chủ phục vụ một tệp nhỏ (ví dụ `/version.json`, `Cache-Control: no-store`) chứa `buildId`. Client so với `RuntimeConfig.buildId` định kỳ và khi tab được đưa lại lên trước
Service Worker (nếu bản web dùng)	Lắng nghe sự kiện phát hiện phiên bản mới, báo khi bản mới đã được tải sẵn
HTTP 426 từ backend	Bản quá cũ: chuyển sang trạng thái bắt buộc

Cả ba đều cho ra cùng `UpdateState`. `install` ở web thực hiện hai móc đầu của mục 3.6 (`flushAutosave`, `settleCompile`) rồi tải lại trang; không có sao lưu hay trình cài. Người dùng thấy cùng một thành phần thông báo, và chỉ phần cách thực hiện khác.

3.8 An toàn dữ liệu khi cập nhật

Mối lo	Biện pháp
Mất thay đổi chưa lưu	Móc `flushAutosave` và chờ xác nhận ghi (mục 3.6)

Mối lo	Biện pháp
Migration IndexedDB hỏng giữa chừng	**Sao lưu trước khi migrate:** tạo bản `.structpaper` (JSON + asset) của mọi bản thảo vào thư mục dữ liệu của ứng dụng (desktop) hoặc kho riêng (web), giữ hai bản gần nhất, dọn sau một thời gian. Migration chạy trong giao dịch; thất bại thì khôi phục từ dữ liệu cũ và đưa ứng dụng về chế độ chỉ đọc kèm đường xuất dữ liệu
Hạ cấp	**Không hỗ trợ.** Nếu mã chạy là bản cũ hơn dữ liệu (phiên bản IndexedDB hoặc `schemaVersion` lớn hơn mức hiểu được) thì từ chối ghi, hiện thông báo cần cập nhật, vẫn cho xuất dữ liệu
Bản cũ ghi đè trường mới	Server từ chối ghi khi client dưới `minWriterVersion` của bản thảo (mục 2.8)
Môi trường biên dịch khác nhau giữa các bản	Khóa `compileCache` đã có `environmentId`, nên cache cũ không bị dùng nhầm; bản thảo ghim môi trường mà app mới không còn thì đề nghị dùng server (mục 2.7)
Cài đặt người dùng đổi định dạng	Có phiên bản cho `settings` và hàm migrate có test
Bộ cài mới xóa dữ liệu	Dữ liệu người dùng nằm ngoài thư mục chương trình; bộ cài và bản cập nhật **không** động đến thư mục dữ liệu; kiểm thử riêng điều này (mục 3.12)

3.9 Bảo mật và đặc thù từng hệ điều hành

Bảo mật của chuỗi cập nhật

Mối đe dọa	Biện pháp
Gói cập nhật bị thay	Chữ ký bắt buộc, xác minh bằng khóa công khai trong ứng dụng; không bao giờ cài khi chữ ký sai (`U2_SIGNATURE`)

Mối đe dọa	Biện pháp
Truyền tải bị nghe lén hoặc sửa	Endpoint chỉ HTTPS; không bật tùy chọn cho phép HTTP
Tấn công hạ cấp	Mặc định updater chỉ cài bản cao hơn bản đang chạy. Endpoint động có thể ép cài một phiên bản chỉ định (để quay lui) nhưng **vẫn phải là gói có chữ ký hợp lệ**; chỉ server do ta kiểm soát mới được làm điều này
Lộ khóa ký	Khóa riêng chỉ ở môi trường phát hành có người duyệt; không có trong PR từ fork; kế hoạch xoay khóa (mục 3.3)
Tag hoặc release giả	Bật bảo vệ tag `v*` (chỉ người có quyền được tạo); bảo vệ nhánh; chỉ workflow có quyền ghi release
Chuỗi cung ứng trong CI	Ghim action theo mã commit; `permissions` tối thiểu cho từng job; khóa phiên bản phụ thuộc bằng lockfile; kiểm tra lỗ hổng phụ thuộc (npm và cargo); lập danh mục thành phần (SBOM); đăng băm SHA-256 của các tệp cùng release
Lộ token trong ứng dụng	Endpoint cập nhật công khai, không nhúng token GitHub hay khóa API vào app
Quyền quá rộng của giao diện	Cấu hình quyền Tauri chỉ cấp updater, khởi động lại và các quyền thật sự cần (mục 2.6)

Đặc thù theo hệ điều hành

Hệ điều hành	Điểm cần chú ý
Windows	Chọn kiểu bộ cài (NSIS hoặc MSI). **Cài theo người dùng hiện tại** không cần quyền quản trị, hợp với máy trường học hạn chế quyền, và cho phép cập nhật không cần nâng quyền; cài theo máy cần quyền quản trị mỗi lần cập nhật. Ký cả tệp thực thi của sidecar (Tectonic) để giảm cảnh báo SmartScreen và phần mềm diệt virus báo nhầm. Cần có WebView2 (bộ cài có thể kèm bước cài đặt nếu thiếu)

Hệ điều hành	Điểm cần chú ý
macOS	Ký bằng chứng chỉ Developer ID, bật hardened runtime và notarize cả ứng dụng lẫn sidecar. Quyết định build riêng cho Intel và Apple Silicon hay build gộp. Bản cập nhật là `.app.tar.gz` có chữ ký
Linux	AppImage dùng được với updater. Gói `deb` hoặc `rpm` không tự cập nhật bằng updater mà đi qua trình quản lý gói; cần quyết định có hỗ trợ không

Các chi tiết về ký mã, notarize và định dạng bản cập nhật cần xác nhận theo phiên bản Tauri đang dùng.

3.10 Giám sát và số liệu

Ghi nhận (không kèm nội dung bài hay thông tin cá nhân), gắn `appVersion`, hệ điều hành, kiến trúc, kênh và `X-Install-Id`:

Sự kiện	Ý nghĩa
`update_check` (kết quả: có hoặc không)	Số người dùng được hỏi
`update_available`	Số người thấy thông báo
`update_download_started`, `update_downloaded`	Tỷ lệ bắt đầu và hoàn tất tải
`update_installed`, `update_relaunched`	Tỷ lệ cài thành công
`update_error{code}`	Lỗi theo mã `U1` đến `U7`
`startup_ok`	Ứng dụng khởi động thành công và chạy ổn quá một ngưỡng thời gian

Từ đó theo dõi: tỷ lệ phân bố phiên bản, thời gian trung bình để người dùng lên bản mới, tỷ lệ lỗi theo hệ điều hành, và **phát hiện sự cố sau cập nhật**: nếu một phiên bản mới liên tiếp không đạt `startup_ok` ở nhiều thiết bị thì cảnh báo và tạm dừng phát hành dần (mục 3.11). Có công tắc tắt thu thập cho người dùng, và chính sách ghi rõ dữ liệu nào được thu thập.

3.11 Phát hành dần, chặn và thu hồi bản lỗi

Tính năng này cần endpoint động (lựa chọn B ở mục 3.4).

Phát hành dần. Mỗi bản có `rolloutPercent` và trạng thái (`draft`, `rolling`, `paused`, `complete`, `blocked`). Tăng từng bậc (ví dụ 5%, 25%, 100%), mỗi bậc chờ đủ thời gian và đạt ngưỡng số liệu (tỷ lệ lỗi cập nhật, tỷ lệ `startup_ok`). Người dùng được xếp vào thùng cố định theo băm của `X-Install-Id`, nên không nhảy qua lại giữa các lần kiểm tra.

Ép nâng cấp. `minClientVersion` chỉ được nâng khi bản cũ thực sự không còn được backend hỗ trợ (mục 2.8), kèm thông báo trước ít nhất một chu kỳ phát hành.

Thu hồi bản lỗi:

Tình huống	Cách xử lý
Bản nháp (chưa publish) có vấn đề	Xóa release nháp và tag, sửa rồi phát hành lại
Đã publish, phát hiện sớm khi ít người đã cập nhật	Với server động: chuyển bản đó sang `paused` hoặc `blocked`, người chưa cập nhật không nhận. Với tệp tĩnh: bỏ nền tảng bị ảnh hưởng khỏi `latest.json` hoặc đưa nó về bản tốt trước
Lỗi nặng ảnh hưởng dữ liệu	Dừng ngay phát hành; phát hành **bản sửa với số phiên bản cao hơn** (không có "hạ cấp" cho người đã cài, vì dữ liệu đã có thể ở schema mới); thông báo trong ứng dụng qua `config/client`
Muốn quay về bản cũ	Về nguyên tắc thực hiện bằng cách phát hành bản sửa tiến lên (*roll-forward*). Người dùng cài tay bản cũ chỉ an toàn khi dữ liệu còn tương thích (mục 3.8)

Đường nhanh cho hotfix: bỏ qua kênh beta nhưng vẫn qua đủ kiểm tra sau build và bước duyệt của con người; đặt mục tiêu thời gian từ khi phát hiện đến khi publish. Trạng thái sự cố có thể được hiển thị cho người dùng bằng thông báo do server điều khiển.

3.12 Kiểm thử cập nhật

Mỗi lần phát hành đều chạy bộ kiểm thử dưới đây (một phần tự động, một phần thủ công trên máy thật hoặc máy ảo trước khi bấm duyệt):

Kịch bản	Cách kiểm	Mong đợi
Cập nhật liền kề (ví dụ 1.0.1 lên 1.0.2)	Cài bản cũ từ bộ cài thật, trỏ endpoint thử, cập nhật	Cài thành công, mở lại bình thường, dữ liệu còn nguyên
Cập nhật nhảy cóc (1.0.0 lên 1.2.0)	Tương tự, qua nhiều bản	Migration chạy tuần tự đúng
Chữ ký sai hoặc gói bị sửa	Cung cấp gói chỉnh sửa	Từ chối cài, báo `U2_SIGNATURE`
Mạng đứt giữa chừng; hết dung lượng	Mô phỏng	Không cài gói dở, cho thử lại, báo đúng mã lỗi
Dữ liệu sau cập nhật	Dùng bộ bài mẫu có ảnh, revision, hàng chờ đồng bộ chưa gửi	Toàn vẹn, không mất
Migration lỗi	Mô phỏng lỗi trong migration	Khôi phục từ sao lưu, chế độ chỉ đọc, xuất dữ liệu được
Bản bắt buộc	Server trả 426 hoặc `minClientVersion` cao	Màn hình bắt buộc, vẫn xuất sao lưu được
Chuyển kênh stable và beta	Đổi kênh trong cài đặt	Nhận đúng bản của kênh
Phát hành dần	Hai `X-Install-Id` năm trong và ngoài thùng	Chỉ thùng trong nhận bản mới
Hạ cấp thủ công	Cài bản cũ lên dữ liệu mới	Từ chối ghi, đề nghị cập nhật
Cài mới rồi cập nhật	Máy sạch	Hoạt động đúng, không phụ thuộc dữ liệu có sẵn
Theo hệ điều hành	Windows theo người dùng và theo máy; macOS đã notarize; Linux AppImage	Cập nhật và mở lại thành công

Trong CI có một **server cập nhật giả** để chạy các kịch bản tự động; bước kiểm tra sau build xác minh chữ ký từng gói bằng khóa công khai. Các kịch bản trên máy thật là mục bắt buộc trong danh sách kiểm tra của bước duyệt (giai đoạn 6 của workflow, mục 3.5).